

CG Viva and Explanations

Computer Graphics Lab Exam Notes

প্রথমে OpenGL/GLUT সম্পর্কে সাধারণ ধারণা

ফাংশন	কাজ
<code>glutInit()</code>	GLUT toolkit initialize করে
<code>glutInitDisplayMode()</code>	Display mode সেট করে (RGB, Single/Double buffer)
<code>glutInitWindowSize()</code>	Window এর সাইজ নির্ধারণ
<code>glutCreateWindow()</code>	Window তৈরি করে
<code>glClearColor()</code>	Background color সেট করে
<code>glColor3f()</code>	Drawing color সেট করে
<code>glFlush()</code>	সব output display-এ পাঠায়
<code>gluOrtho2D()</code>	2D coordinate system সেট করে

প্রায় সব প্রোগ্রামের basic flow একই:

- Header include করা হয়: `windows.h`, `GL/gl.h`, `GL/glu.h`, `GL/glut.h`
- `main()` এ GLUT initialize করা হয়:
 - `glutInit(&argc, argv)` toolkit initialize করে।
 - `glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)` single buffer ও RGB color mode সেট করে।
 - `glutInitWindowSize(width, height)` window size সেট করে।
 - `glutInitWindowPosition(x, y)` screen-এ window position সেট করে।
 - `glutCreateWindow("title")` window তৈরি করে।
- `myInit()` বা `init()` function:
 - `glClearColor(r,g,b,a)` background color সেট করে।
 - `glColor3f(r,g,b)` drawing color সেট করে।

- `glMatrixMode(GL_PROJECTION)` projection matrix select করে।
 - `glLoadIdentity()` current matrix reset করে।
 - `gluOrtho2D(left,right,bottom,top)` 2D coordinate system সেট করে।
4. `glutDisplayFunc(display)` display callback register করে।
 5. `glutMainLoop()` event loop চালু করে।

Important note: `glColor3f()` -এ color value সাধারণত `0.0` থেকে `1.0` এর মধ্যে হওয়া উচিত। PDF-এর কিছু code-এ `255.0f` আছে, exam-এ concept বললে `1.0f` বলা better।

1. Draw Some Points

উদ্দেশ্য

OpenGL ব্যবহার করে নির্দিষ্ট coordinate-এ point draw করা।

মূল ধারণা

Point draw করতে `glBegin(GL_POINTS)` এবং `glEnd()` এর মধ্যে `glVertex2i(x,y)` ব্যবহার করা হয়।

Step by Step

1. Background white করা হয়েছে:

```
glClearColor(1.0, 1.0, 1.0, 0.0)
```

2. Drawing color black:

```
glColor3f(0.0f, 0.0f, 0.0f)
```

3. Coordinate system:

```
gluOrtho2D(0.0, 640.0, 0.0, 480.0)
```

অর্থাৎ x-axis 0 থেকে 640, y-axis 0 থেকে 480।

4. Display function-এ screen clear:

```
glClear(GL_COLOR_BUFFER_BIT)
```

5. Point size:

```
glPointSize(20.0)
```

6. Four points draw:

- (20,10)
- (50,10)
- (20,80)
- (50,80)

7. `glFlush()` output display-তে পাঠায়।

মনে রাখার বিষয়

`GL_POINTS` primitive একেকটি vertex-কে আলাদা point হিসেবে draw করে।

2. Line Drawing Using Slope Intercept Method

উদ্দেশ্য

Line equation $y = mx + b$ ব্যবহার করে point by point line draw করা।

Formula

Slope:

$$m = (y_2 - y_1) / (x_2 - x_1)$$

Intercept:

$$b = y_1 - m * x_1$$

যদি y value বাড়িয়ে x বের করতে চাই:

$$x = (y - b) / m$$

Code-এর Example

$$x_1 = 50, y_1 = 100$$

```
x2 = 100, y2 = 200
```

তাই:

```
m = (200 - 100) / (100 - 50) = 2  
b = 100 - 2*50 = 0
```

Step by Step

1. Initial point `(x1,y1)` নেওয়া হয়।
2. Slope `m` calculate করা হয়।
3. Intercept `b` calculate করা হয়।
4. First point draw করা হয়।
5. Loop চলে যতক্ষণ `y < y2` :
 - `y++`
 - `x = round((y-b)/m)`
 - point draw করা হয়।

কেন `floor(xf + 0.5)` ?

এটি floating point value-কে nearest integer-এ convert করে। কারণ screen pixel coordinate integer হয়।

Limitation

Slope intercept method floating point multiplication/division ব্যবহার করে, তাই Bresenham-এর তুলনায় slower।

3. Line Drawing Using Bresenham Algorithm

উদ্দেশ্য

Integer arithmetic ব্যবহার করে efficient line drawing করা।

মূল ধারণা

Bresenham algorithm প্রতিটি step-এ decide করে next pixel হবে:

```
E = (x+1, y)
NE = (x+1, y+1)
```

Decision parameter `d` দিয়ে ঠিক করা হয় কোন pixel line-এর কাছাকাছি।

PDF Code-এর Variables

```
dx = x2 - x1;
dy = y2 - y1;
inc1 = 2 * dy;
inc2 = 2 * (dy - dx);
d = inc1 - dx;
```

Step by Step

1. Start point:

```
x = x1, y = y1
```

2. `dx` ও `dy` বের করা হয়।

3. Decision parameter initialize:

```
d = 2*dy - dx
```

4. যতক্ষণ `x <= x2`:

- current point `(x,y)` draw
- `x++`
- যদি `d < 0`, line x-axis-এর দিকে কাছাকাছি:

```
d = d + 2*dy
```

- না হলে y-ও বাড়ে:

```
y++d = d + 2*(dy-dx)
```

Important Limitation

এই simple code সাধারণত `0 <= slope <= 1` line-এর জন্য ঠিক। PDF-এর point `(50,100)` থেকে `(100,200)` হলে slope `2`, তাই generalized Bresenham হলে steep slope handle

করার জন্য x/y swap বা y-major loop দরকার।

কেন Bresenham fast?

কারণ এটি floating point নয়, integer addition/subtraction ব্যবহার করে।

4. Translation Transformation

উদ্দেশ্য

একটি triangle-কে এক position থেকে অন্য position-এ সরানো।

Formula

$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

Matrix:

```
[1 0 tx]
[0 1 ty]
[0 0 1 ]
```

Code Flow

1. `glOrtho(-50,50,-50,50,-1,1)` দিয়ে coordinate range সেট।
2. `draw_triangle()` তিনটি vertex দিয়ে triangle draw করে:
 - `(0,25)`
 - `(25,-25)`
 - `(-25,-25)`
3. প্রথমে original triangle draw করা হয়।
4. `glEnable(GL_LINE_STIPPLE)` dotted line style enable করে।
5. `glTranslatef(-20.0, 0.0, 0.0)` triangle-কে x-axis বরাবর 20 unit left shift করে।
6. আবার `draw_triangle()` call করলে translated triangle দেখা যায়।

মনে রাখার বিষয়

`glLoadIdentity()` দিলে আগের transformation reset হয়।

5. Rotation Transformation

উদ্দেশ্য

Triangle-কে নির্দিষ্ট angle-এ rotate করা।

Formula

Origin around rotation:

$$\begin{aligned}x' &= x \cos \theta - y \sin \theta \\y' &= x \sin \theta + y \cos \theta\end{aligned}$$

Matrix:

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Code Flow

1. Original triangle draw করা হয়।
2. `glEnable(GL_LINE_STIPPLE)` dotted transformed triangle দেখানোর জন্য।
3. `glRotatef(90.0, 0.0, 0.0, 1.0)` ব্যবহার করা হয়।
4. এর অর্থ:
 - Angle = 90 degree
 - Rotation axis = z-axis
 - 2D object xy-plane-এ থাকে, তাই z-axis around rotation হয়।
5. আবার triangle draw করলে rotated triangle দেখা যায়।

Important

Positive angle OpenGL-এ সাধারণত counter-clockwise rotation দেয়।

6. Circle Drawing Using Bresenham/Midpoint Algorithm

উদ্দেশ্য

Circle draw করা using integer decision parameter and symmetry।

Circle Equation

$$(x - h)^2 + (y - k)^2 = r^2$$

Center:

$$(h, k) = (200, 200)$$

Radius:

$$r = 100$$

Initial Values

```
x = 0;  
y = r;  
d = 3 - 2*r;
```

8-Way Symmetry

Circle symmetric, তাই এক octant-এর point থেকে ৮টি point draw করা যায়:

```
( h+x, k+y )  
( h+x, k-y )  
( h+y, k+x )  
( h+y, k-x )
```



```
( h-y, k+x )
( h-y, k-x )
( h-x, k+y )
( h-x, k-y )
```

Step by Step

1. Start point $(0, r)$ ।
2. Loop চলে $x \leq y$ পর্যন্ত।
3. 8 symmetric points draw করা হয়।
4. যদি $d < 0$:
 - East pixel choose
 - $d = d + 4*x + 6$
5. না হলে:
 - South-East pixel choose
 - $d = d + 4*(x-y) + 10$
 - $y--$
6. প্রতি iteration-এ $x++$ ।

কেন only $x \leq y$?

কারণ এক octant complete হলে symmetry দিয়ে পুরো circle পাওয়া যায়।

7. Scaling Transformation

উদ্দেশ্য

Triangle-এর size x বা y direction-এ বড়/ছোট করা।

Formula

$$\begin{aligned}x' &= s_x * x \\ y' &= s_y * y\end{aligned}$$

Matrix:

```
[sx 0 0]
[0 sy 0]
[0 0 1]
```

Code Flow

1. Original triangle draw করা হয়।
2. Dotted style enable করা হয়।
3. `glScalef(1.0, 0.5, 1.0)` call করা হয়।
4. অর্থ:
 - x direction unchanged
 - y direction half
 - z unchanged
5. আবার triangle draw করলে y-axis বরাবর compressed triangle দেখা যায়।

Important

Scaling origin `(0,0)` relative হয়। Object origin থেকে দূরে থাকলে scale করার সময় position-ও visually change হতে পারে।

8. Mouse Operation

উদ্দেশ্য

Mouse event handle করে different shapes draw করা।

Main Callback

```
glutMouseFunc(myMouse);
```

এটি mouse button press/release হলে `myMouse()` call করে।

`myMouse()` Parameters

```
void myMouse(int button, int state, int x, int y)
```

- `button` : কোন button চাপা হয়েছে।
- `state` : press নাকি release।
- `x, y` : mouse cursor position।

Code Behavior

1. Left button down হলে:

- Green rectangle draw:

```
glRecti(200,150,500,330)
```

2. Right button down হলে:

- Red circle draw using midpoint circle logic।

Important

`GLUT_LEFT_BUTTON` , `GLUT_RIGHT_BUTTON` , `GLUT_DOWN` এগুলো GLUT predefined constants।

9. Arm Motion

উদ্দেশ্য

Hierarchical transformation ব্যবহার করে robotic arm motion দেখানো।

Main Concept

Arm-এর দুই অংশ:

1. Shoulder segment
2. Elbow segment

Shoulder rotate করলে elbow segment-ও follow করে, কারণ elbow shoulder-এর child transformation।

Important Functions

- `glPushMatrix()` current matrix save করে।

- `glPopMatrix()` saved matrix restore করে।
- `glTranslatef()` object সরায়।
- `glRotatef()` object ঘোরায়।
- `glScalef()` cube-কে rectangular arm segment বানায়।
- `glutWireCube(1.0)` wireframe cube draw করে।

Code Flow

1. Shoulder joint origin set করার জন্য translate করা হয়।
2. `glRotatef(shoulder,0,0,1)` shoulder rotate করে।
3. First arm segment draw:
 - cube scale করে long rectangle বানানো হয়।
4. Elbow position-এ translate করা হয়।
5. `glRotatef(elbow,0,0,1)` elbow rotate করে।
6. Second arm segment draw করা হয়।

Keyboard Control

```
s : shoulder +5 degree
S : shoulder -5 degree
e : elbow +5 degree
E : elbow -5 degree
Esc : exit
```

`glutPostRedisplay()` display function আবার call করতে বলে।

কেন `GLUT_DOUBLE` ?

Animation smooth করার জন্য double buffering ব্যবহার করা হয়েছে। তাই display শেষে

`glutSwapBuffers()` ।

10. Ellipse Drawing

উদ্দেশ্য

Ellipse draw করা।

Ellipse Equation

$$x^2/a^2 + y^2/b^2 = 1$$

From this:

$$y = b * \sqrt{1 - x^2/a^2}$$

Code Values

```
center = (h,k) = (300,300)
a = 150
b = 25
```

Here:

- **a** = x-radius / major radius
- **b** = y-radius / minor radius

Step by Step

1. **x** from **1** to **a** loop করে।
2. Formula দিয়ে **y** বের করে:
$$y = b * \sqrt{1 - x^2/a^2}$$
3. Four symmetric points draw:
 - **(h+x, k+y)**
 - **(h-x, k-y)**
 - **(h-x, k+y)**
 - **(h+x, k-y)**

Important Note

PDF title-এ Bresenham বলা হলেও code-টি actual Bresenham ellipse নয়; এটি direct equation method, কারণ এখানে `sqrt()` এবং `pow()` ব্যবহার হয়েছে।

11. Polygon Drawing

উদ্দেশ্য

Multiple vertex ব্যবহার করে polygon draw করা।

Main Primitive

```
glBegin(GL_POLYGON);
```

`GL_POLYGON` সব vertex connect করে filled polygon তৈরি করে।

Code Flow

1. Background white, drawing setup।

2. `glBegin(GL_POLYGON)` ।

3. Red color:

```
glColor3f(1.0,0.0,0.0)
```

4. Several vertices দেওয়া হয়েছে:

- `(100,50)`
- `(200,130)`
- `(200,70)`
- `(200,130)`
- `(500,170)`
- `(300,130)`

5. `glEnd()` polygon finish করে।

Important

`GL_POLYGON` convex polygon-এর জন্য reliable। Concave বা self-intersecting polygon হলে result unpredictable হতে পারে।

12. Line Triangle Transformation

উদ্দেশ্য

Line এবং triangle draw করে translation transformation দেখানো।

Code Structure

1. `triangle()` :
 - Magenta triangle draw করে।
 - Vertices:
`(50,150)` , `(50,250)` , `(130,200)`
2. `line()` :
 - Yellow line draw করে।
 - Start `(sx,sy)` , end `(dx,dy)` ।
3. `display()` :
 - First original line and triangle draw:
`line(100,150,100,250);triangle();`
 - Then transformation:
`glTranslatef(100.0,50.0,0.0);`
 - Same line and triangle আবার draw করলে translated copy দেখা যায়।

Formula

$$\begin{aligned}x' &= x + 100 \\y' &= y + 50\end{aligned}$$

Important Viva Questions and Answers

Basic OpenGL

সংক্ষিপ্ত প্রশ্ন (Short Questions)

`glBegin()` ফাংশনে ব্যবহৃত primitive types গুলো কী কী?

উত্তর:

Primitive	কাজ
<code>GL_POINTS</code>	বিন্দু আঁকে
<code>GL_LINES</code>	দুটি vertex দিয়ে line
<code>GL_LINE_LOOP</code>	বন্ধ রেখা (outline)
<code>GL_TRIANGLES</code>	ভরা ত্রিভুজ
<code>GL_POLYGON</code>	ভরা বহুভুজ

Q1. GLUT কী?

A: GLUT হলো OpenGL Utility Toolkit। এটি window create, keyboard/mouse event handle এবং display callback manage করতে সাহায্য করে।

Q2. `glutMainLoop()` কী করে?

A: এটি event processing loop চালু করে। Window বন্ধ না হওয়া পর্যন্ত display, mouse, keyboard callback handle করে।

Q3. `glClearColor()` ও `glClear()` এর পার্থক্য কী?

A: `glClearColor()` background clear করার color সেট করে। `glClear(GL_COLOR_BUFFER_BIT)` সেই color দিয়ে screen clear করে।

Q4. `glBegin()` এবং `glEnd()` কেন ব্যবহার হয়?

A: OpenGL primitive drawing শুরু ও শেষ করতে। যেমন `GL_POINTS`, `GL_LINES`, `GL_POLYGON`।

Q5. `glVertex2i()` কী করে?

A: এটি একটি 2D vertex দেয়, যেখানে coordinate integer।

Q6. `glFlush()` কেন লাগে?

A: OpenGL command buffer-এর command display-তে execute করতে বাধ্য করে।

Q7. `gluOrtho2D()` কী?

A: এটি 2D orthographic projection set করে; অর্থাৎ window coordinate range define করে।

Q8. `glMatrixMode(GL_PROJECTION)` এবং `GL_MODELVIEW` পার্থক্য কী?

A: `GL_PROJECTION` camera/projection coordinate set করে, আর `GL_MODELVIEW` object transformation যেমন translate, rotate, scale handle করে।

Line Drawing

Q9. Slope intercept formula কী?

A: $y = mx + b$, যেখানে $m = (y_2 - y_1) / (x_2 - x_1)$ এবং $b = y_1 - m * x_1$ ।

Q10. Slope intercept method-এর অসুবিধা কী?

A: Floating point calculation লাগে, তাই slow এবং rounding error হতে পারে।

Q11. Bresenham line algorithm কেন efficient?

A: এটি only integer addition/subtraction ব্যবহার করে, multiplication/division বা floating point লাগে না।

Q12. Bresenham line-এর decision parameter কী করে?

A: Next pixel east হবে নাকি north-east হবে তা decide করে।

Q13. Simple Bresenham code কোন slope-এর জন্য বেশি suitable?

A: সাধারণ simple version $0 \leq m \leq 1$ line-এর জন্য। অন্য slope handle করতে generalized version দরকার।

Transformation

Q14. Translation কী?

A: Object-কে এক position থেকে অন্য position-এ shift করা। Formula: $x' = x + tx$, $y' = y + ty$ ।

Q15. Rotation কী?

A: Object-কে কোনো point বা axis-এর around নির্দিষ্ট angle-এ ঘোরানো।

Q16. 2D rotation matrix কী?

A:

```
[cos theta  -sin theta]
[sin theta   cos theta]
```

Q17. Scaling কী?

A: Object-এর size বড় বা ছোট করা। Formula: $x' = sx * x$, $y' = sy * y$ ।

Q18. `glLoadIdentity()` কেন ব্যবহার করা হয়?

A: Current matrix reset করে, যাতে আগের transformation নতুন drawing-এ unwanted effect না করে।

Q19. `glTranslatef(tx,ty,tz)` কী করে?

A: Object বা coordinate system-কে x, y, z direction-এ translate করে।

Q20. `glRotatef(angle,x,y,z)` -এ x,y,z কী?

A: Rotation axis define করে। 2D rotation-এর জন্য সাধারণত `(0,0,1)` use করা হয়।

Q21. `glScalef(sx,sy,sz)` কী করে?

A: Object-কে x, y, z direction-এ scale করে।

Circle and Ellipse

Q22. Circle equation কী?

A: $(x-h)^2 + (y-k)^2 = r^2$, যেখানে `(h,k)` center এবং `r` radius।

Q23. Circle drawing-এ 8-way symmetry কেন ব্যবহার হয়?

A: Circle symmetric; এক octant-এর point থেকে ৮টি point পাওয়া যায়, তাই calculation কমে।

Q24. Midpoint circle algorithm-এর initial decision parameter কী?

A: Common form: $d = 3 - 2r$ ।

Q25. $x \leq y$ পর্যন্ত loop কেন চলে?

A: কারণ first octant complete হলেই symmetry দিয়ে পুরো circle draw করা যায়।

Q26. Ellipse equation কী?

A: $x^2/a^2 + y^2/b^2 = 1$ ।

Q27. PDF-এর ellipse code কি সত্যিকারের Bresenham?

A: না। এখানে `sqrt()` ও `pow()` ব্যবহার করা হয়েছে, তাই এটি direct equation method। Bresenham/Midpoint ellipse হলে decision parameter ব্যবহার করা হয়।

Mouse and Interaction

Q28. Mouse event handle করতে কোন function register করা হয়?

A: `glutMouseFunc(myMouse)`।

Q29. `myMouse(int button, int state, int x, int y)` -এ parameters কী?

A: `button` কোন mouse button, `state` press/release, আর `x,y` mouse position।

Q30. `GLUT_DOWN` কী বোঝায়?

A: Mouse button press event।

Arm Motion

Q31. Hierarchical transformation কী?

A: Parent object transform হলে child object-ও তার সাথে transform হয়। Robotic arm-এ shoulder parent, elbow child।

Q32. `glPushMatrix()` ও `glPopMatrix()` কেন লাগে?

A: Transformation state save ও restore করার জন্য, যাতে এক object-এর transformation অন্য object-এ unwantedভাবে না লাগে।

Q33. `glutPostRedisplay()` কী করে?

A: Display callback আবার call করার request করে, ফলে updated drawing দেখা যায়।

Q34. `GLUT_DOUBLE` ও `glutSwapBuffers()` কেন ব্যবহার হয়?

A: Smooth animation-এর জন্য double buffering। Back buffer-এ draw হয়, তারপর front buffer-এর সাথে swap হয়।

Polygon

Q35. `GL_POLYGON` কী করে?

A: দেওয়া vertices connect করে filled polygon draw করে।

Q36. `GL_LINE_LOOP` ও `GL_POLYGON` পার্থক্য কী?

A: `GL_LINE_LOOP` শুধু boundary line draw করে, `GL_POLYGON` filled area draw করে।

Q37. `GL_LINES` কীভাবে কাজ করে?

A: প্রতি দুইটি vertex মিলে একটি line segment তৈরি করে।